数据结构：第六章排序算法
复习讲义

目录

1 排序预备知识 (Preliminaries) 2
1.1 通用排序接口 . . . 2
1.2 排序算法基本术语 . . . 2

2 插入排序 (Insertion Sort) 2
2.1 算法思想 . . . 2
2.2 代码实现 . . . 3
2.3 复杂度分析 . . . 3
2.4 特点总结 . . . 4

3 选择排序 (Selection Sort) 4
3.1 算法思想 . . . 4
3.2 代码实现 . . . 4
3.3 复杂度分析 . . . 5
3.4 选择排序特点总结 . . . 5

4 简单排序算法的下界 (Lower Bound for Simple Sorting) 5
4.1 逆序对 (Inversions) . . . 5
4.2 逆序对与插入排序的关系 . . . 6
4.3 对” 接近有序” 输入的优势 . . . 6
4.4 简单排序算法的下界定理 . . . 6
4.5 下界定理的意义与启发 . . . 6

5 希尔排序 (Shellsort) 7
5.1 算法框架 . . . 7
5.2 代码实现（Shell 增量序列） . . . 7
5.3 排序过程示例 . . . 7
5.4 增量序列分析 . . . 8
5.5 希尔排序特点总结 . . . 9

6 堆排序 (Heapsort) 9
6.1 算法一：借助辅助数组 . . . 9
6.2 算法二：原地堆排序 . . . 10
6.3 复杂度分析 . . . 10
6.4 堆排序特点总结 . . . 11

7 归并排序 (Mergesort) 11
7.1 步骤一：合并两个有序表 . . . 11
7.2 步骤二：递归归并排序（MSort） . . . 12
7.3 步骤三：复杂度分析 . . . 13
7.4 迭代版本 (Iterative Mergesort) . . . 13
7.5 归并排序特点总结 . . . 15

8 快速排序 (Quicksort) 16
8.1 算法概述 . . . 16
8.2 基准元素选取 (Picking the Pivot) . . . 16
8.3 分区策略 (Partitioning Strategy) . . . 17
8.4 小数组优化 . . . 17
8.5 完整代码实现 . . . 17
8.6 复杂度分析 . . . 19
8.7 选择问题 (The Selection Problem) . . . 19
8.8 快速排序特点总结 . . . 20

9 大结构排序 (Table Sort) 20
9.1 问题背景 . . . 20
9.2 物理重排（利用置换环） . . . 20
9.3 复杂度 . . . 20

10 排序算法的通用下界 (General Lower Bound for Sorting) 21
10.1 决策树证明 . . . 21
10.2 与第三节逆序对下界的关系 . . . 21

11 桶排序与基数排序 (Bucket Sort & Radix Sort) 22
11.1 桶排序 (Bucket Sort) . . . 22
11.2 基数排序 (Radix Sort) . . . 22
11.3 桶排序与基数排序特点总结 . . . 23

12 全章排序算法横向对比 24

13 全章总结 24

1 排序预备知识 (Preliminaries)

1.1 通用排序接口

课件给出的排序函数原型为：

```
void X_Sort ( ElementType A[ ],  int N )
```

课件对本章默认的假设如下：

- N 是合法整数。
- 为简单起见，考虑整数数组。
- **基于比较的排序 (Comparison-based sorting)**：只允许对元素做比较和交换操作（$<, >, \leq, \geq$），不利用元素的具体数值（如计数排序）。
- **内部排序 (Internal Sorting)**：整个数组可以放入主存中，无需外部存储。

1.2 排序算法基本术语

- **原地排序 (In-place)**：只使用 $O(1)$ 的额外内存空间（不含递归栈），不需要辅助数组。
- **稳定排序 (Stable)**：相等的元素在排序后保持原有的相对顺序。
- **内部排序 vs 外部排序**：数据全部在内存中的为内部排序；数据量超过内存大小、需要磁盘辅助的为外部排序。

2 插入排序 (Insertion Sort)

2.1 算法思想

插入排序的核心思想类似于打牌时整理手牌：每次取出一张新牌（未排序部分的第一个元素），在已排好序的牌中找到合适的位置，把它插入进去。

具体步骤为：

1. 从第 2 个元素（下标 $P = 1$）开始，逐一处理每个元素。
2. 将当前元素 `Tmp = A[P]` 暂存。
3. 将已排序部分中所有大于 `Tmp` 的元素向右移动一位，腾出位置。
4. 将 `Tmp` 放入空出的位置。

2.2 代码实现

```
void InsertionSort ( ElementType A[ ], int N )
{
    int  j, P;
    ElementType  Tmp;

    for ( P = 1; P < N; P++ ) {
        Tmp = A[ P ];         /* 取出下一张待插入的牌 */
        for ( j = P; j > 0 && A[ j - 1 ] > Tmp; j-- )
            A[ j ] = A[ j - 1 ]; /* 已排序部分向右移位 */
        A[ j ] = Tmp;         /* 将 Tmp 插入正确位置 */
    }
}
```

代码关键点

- 外层循环：P 从 1 到 $N-1$，逐步扩大已排序前缀。
- 内层循环：将比 `Tmp` 大的元素整体右移，不是逐一交换。每次移动一个元素只相当于”搬运”，不计为交换（swap），这与后文逆序对分析中的”交换相邻元素”等价。
- 内层循环结束时，`j` 指向空出位置，直接赋值 `A[j] = Tmp`。

2.3 复杂度分析

最坏情况 (Worst Case)

当输入数组为逆序时，情况最坏。对于 $P = k$，内层循环需要移动 k 个元素：

$$T(N) = \sum_{P=1}^{N-1} P = \frac{N(N-1)}{2} = O(N^2).$$

最优情况 (Best Case)

当输入数组已经有序时，内层循环条件 `A[j-1] > Tmp` 立刻不满足，每次直接跳过：

$$T(N) = O(N).$$

平均情况 (Average Case)

对随机输入，平均每次插入需要移动约 $P/2$ 个元素，总体仍为：

$$T(N) = O(N^2).$$

2.4 特点总结

性质	结论	原因
时间复杂度（最坏）	$O(N^2)$	逆序输入，每次都移动全部已排序部分
时间复杂度（最优）	$O(N)$	已排序输入，内层循环从不执行
空间复杂度	$O(1)$	只用 `Tmp` 一个临时变量，原地排序
稳定性	稳定	只移动严格大于的元素，相等元素不移位

3 选择排序 (Selection Sort)

3.1 算法思想

选择排序的核心思路：每次从**未排序部分**找出最小元素，将其与未排序部分的第一个元素交换，逐步扩大已排序前缀。

具体步骤为：

1. 外层循环 i 从 0 到 $N-2$，每轮确定第 i 个最小元素的位置。
2. 在 $[i, N-1]$ 范围内扫描，找到最小元素的下标 `MinPos`。
3. 若 `MinPos` $\neq$ `i`，则将 `A[i]` 与 `A[MinPos]` 交换。

3.2 代码实现

```
void SelectionSort ( ElementType A[ ], int N )
{
    int  i, j, MinPos;
    ElementType  Tmp;

    for ( i = 0; i < N - 1; i++ ) {
        MinPos = i;
        for ( j = i + 1; j < N; j++ )
            if ( A[ j ] < A[ MinPos ] )
                MinPos = j;
        if ( MinPos != i ) {
            Tmp = A[ i ];
            A[ i ] = A[ MinPos ];
            A[ MinPos ] = Tmp;
        }
    }
}
```

3.3 复杂度分析

- **比较次数**：无论输入如何，内层循环总是完整扫描剩余部分：

$$\sum_{i=0}^{N-2}(N-1-i)=\frac{N(N-1)}{2}=O(N^2)$$

  因此**最优、平均、最坏情况均为** $O(N^2)$（与输入顺序无关）。

- **交换次数**：至多 $N-1$ 次（每轮最多一次交换），这是选择排序的核心优势。

与插入排序的对比

特性	插入排序	选择排序
时间（最优）	$O(N)$（已有序）	$O(N^2)$（始终扫全）
时间（最坏）	$O(N^2)$	$O(N^2)$
交换次数	最多 $O(N^2)$ 次移位	至多 $N-1$ 次交换
稳定性	稳定	**不稳定**

选择排序在**写操作代价远高于读操作**的场景（如闪存）中有优势，因为它将写次数（交换）压到最低。但对”接近有序”的输入毫无优化，不如插入排序灵活。

3.4 选择排序特点总结

性质	结论	原因
时间复杂度	$O(N^2)$（所有情况）	内层循环始终完整扫描，与输入无关
交换次数	$\leq N-1$ 次	每轮至多一次交换
空间复杂度	$O(1)$	原地排序，只用 `Tmp`
稳定性	**不稳定**	交换可将相等元素跳过其他相等元素

4 简单排序算法的下界 (Lower Bound for Simple Sorting)

4.1 逆序对 (Inversions)

逆序对定义

对于数组 A，若 $i<j$ 但 $A[i]>A[j]$，则称 $(A[i],A[j])$ 是一对**逆序对 (inversion)**。

例如，数组 $[34,8,64,51,32,21]$（课件示例）中，$(34,8)$、$(34,32)$、$(34,21)$、$(64,51)$、$(64,32)$、$(64,21)$、$(51,32)$、$(51,21)$、$(32,21)$ 共 9 对逆序对。

4.2 逆序对与插入排序的关系

核心定理

用插入排序需要恰好 I 次相邻元素交换，其中 I 是数组的逆序对数。
原因：交换一对相邻的、位于错误顺序的元素，恰好消除一个逆序对，且不改变其他任何逆序对。

因此，插入排序的时间复杂度可以更精确地表达为：

$$T(N, I) = O(I + N),$$

其中 I 为原数组的逆序对数，N 为数组长度（N 来自外层循环的开销）。

4.3 对”接近有序”输入的优势

由于 $T(N, I) = O(I + N)$，当输入数组”几乎有序”（即逆序对数 I 很小）时，插入排序表现极佳：

- 若 $I = O(N)$，则 $T = O(N)$，插入排序为线性时间。
- 这是插入排序适合在接近有序的数据上使用的核心原因。

4.4 简单排序算法的下界定理

定理（简单排序下界）

对 N 个不同数的随机排列，其平均逆序对数为：

$$\frac{N(N-1)}{4}.$$

因此，任何仅通过交换相邻元素来排序的算法，其时间复杂度在平均情况下都不低于 $\Omega(N^2)$。

直觉证明：对于 N 个元素的所有 $N!$ 种排列，考虑每一对元素 (i, j)（共 $\binom{N}{2}$ 对）。对这些排列中的每个，元素对是逆序的概率恰好为 $1/2$。因此平均逆序对数为 $\binom{N}{2}/2 = N(N-1)/4$，即 $\Theta(N^2)$。

4.5 下界定理的意义与启发

突破下界的关键思路

想要突破 $O(N^2)$ 下界，**必须在一次交换中消除不止一个逆序对**。
这意味着需要交换距离较远的元素（而不只是相邻元素）。这就是 Shell Sort（希尔排序）、Merge Sort（归并排序）、Heap Sort（堆排序）、Quick Sort（快速排序）等 $O(N \log N)$ 算法的共同出发点：它们都能在一次操作中消除多个逆序对。

5 希尔排序 (Shellsort)

5.1 算法框架

希尔排序定义

1. 定义一个**增量序列 (increment sequence)** $h_1 < h_2 < \cdots < h_t$，且 $h_1 = 1$。
2. 对 $k = t, t-1, \ldots, 1$，依次执行一次 **h_k-排序 (h_k-sort)**：即对所有间隔为 h_k 的子序列分别执行插入排序。

关键性质：经过 h_k-排序后的序列，在随后的 h_{k-1}-排序之后，h_k-有序性仍然保持。

5.2 代码实现（Shell 增量序列）

```
void Shellsort( ElementType A[ ], int N )
{
    int  i, j, Increment;
    ElementType  Tmp;
    for ( Increment = N / 2; Increment > 0; Increment /= 2 )
        /* 对每个增量，执行插入排序 */
        for ( i = Increment; i < N; i++ ) {
            Tmp = A[ i ];
            for ( j = i; j >= Increment; j -= Increment )
                if ( Tmp < A[ j - Increment ] )
                    A[ j ] = A[ j - Increment ];
                else
                    break;
            A[ j ] = Tmp;
        }
}
```

代码关键点

- 外层循环按增量序列从大到小逐步减半：$\lfloor N/2 \rfloor, \lfloor N/4 \rfloor, \ldots, 1$。
- 内两层循环本质上是以步长 `Increment` 做插入排序：`Tmp` 暂存当前元素，逐步向左跳跃比较并移位，最终插入正确位置。
- 当 `Increment = 1` 时退化为普通插入排序，但此时序列已经”接近有序”，操作次数很少。

5.3 排序过程示例

以数组 $[81, 94, 11, 96, 12, 35, 17, 95, 28, 58, 41, 75]$（$N = 12$）为例，Shell 增量序列为 $6, 3, 1$：

阶段	数组状态（示意）
初始	81 94 11 96 12 35 17 95 28 58 41 75
6-sort 后	17 41 11 58 12 35 81 94 28 96 75 (略)
3-sort 后	更接近有序的状态
1-sort 后	11 12 17 28 35 41 58 75 81 94 95 96

5.4 增量序列分析

5.4.1 Shell 增量序列

$$h_t = \left\lfloor \frac{N}{2} \right\rfloor, \quad h_k = \left\lfloor \frac{h_{k+1}}{2} \right\rfloor$$

定理（Shell 增量最坏情况）

使用 Shell 增量序列时，希尔排序的最坏时间复杂度为 $\Theta(N^2)$。

坏输入构造：当 N 是 2 的幂（如 $N = 16$），增量序列为 $8, 4, 2, 1$。设奇数位置存放较大数，偶数位置存放较小数。在 8-sort、4-sort、2-sort 各阶段，奇偶位置的元素始终分属不同子序列，从未相互比较——因为相邻增量均有公因子 2。直到最后的 1-sort，才需要处理大量逆序，退化为 $O(N^2)$。

根本原因

Shell 增量的相邻两项总存在公因子（均为偶数，直到最后的 1），导致较小的增量对更大增量留下的无序性**几乎没有改善效果**。选择增量序列时，相邻增量互质 (relatively prime) 至关重要。

5.4.2 Hibbard 增量序列

$$h_k = 2^k - 1 \quad (1, 3, 7, 15, \ldots)$$

相邻增量**无公因子**（$2^k - 1$ 与 $2^{k-1} - 1$ 互质），从根本上避免了 Shell 增量的缺陷。

定理（Hibbard 增量最坏情况）

使用 Hibbard 增量序列时，希尔排序的最坏时间复杂度为 $\Theta(N^{3/2})$。

猜想（平均情况）：$T_{\text{avg,Hibbard}}(N) = O(N^{5/4})$（尚未被严格证明）。

5.4.3 Sedgewick 增量序列

$$\{1, 5, 19, 41, 109, \ldots\}$$

其中各项形如 $9 \cdot 4^i - 9 \cdot 2^i + 1$ 或 $4^i - 3 \cdot 2^i + 1$（两式交替生成）。

Sedgewick 序列性能（目前最优实用序列）

$$T_{\text{avg}}(N) = O(N^{7/6}), \quad T_{\text{worst}}(N) = O(N^{4/3})$$

5.5 希尔排序特点总结

增量序列	最坏情况	平均情况（猜想）	备注
Shell ($\lfloor N/2^k \rfloor$)	$\Theta(N^2)$	—	相邻增量有公因子，效果差
Hibbard ($2^k - 1$)	$\Theta(N^{3/2})$	$O(N^{5/4})$	互质，有改善
Sedgewick	$O(N^{4/3})$	$O(N^{7/6})$	目前最优实用序列

实用性说明

- 希尔排序算法**极其简单**，但其理论分析**极其复杂**。
- 适合对**中等规模**的输入排序（数万量级）。
- **原地排序**（In-place）：空间复杂度 $O(1)$。
- **不稳定**：h_k-sort 可能使相等元素的相对顺序改变。

6 堆排序 (Heapsort)

6.1 算法一：借助辅助数组

直接利用优先队列（最小堆）的 BuildHeap 和 DeleteMin 操作：

```
Algorithm 1:
{
    BuildHeap( H );                          /* O(N) */
    for ( i = 0; i < N; i++ )
        TmpH[ i ] = DeleteMin( H );          /* O(log N) each */
    for ( i = 0; i < N; i++ )
        H[ i ] = TmpH[ i ];                  /* O(1) each */
}
```

- 时间复杂度：$T(N) = O(N \log N)$。

- 空间复杂度：需要额外 $O(N)$ 辅助数组 TmpH，**空间翻倍**。

6.2 算法二：原地堆排序

避免辅助数组，利用**最大堆**原地完成排序。

算法思路

1. **建堆阶段 (BuildHeap)**：从 $\lfloor N/2 \rfloor$ 向下调用 PercDown，将数组调整为最大堆。完成后 $A[0]$ 为最大元素。
2. **排序阶段**：共 $N-1$ 次循环，每次将堆顶（最大值）与当前堆末元素交换，然后对堆顶执行 PercDown，使堆大小减 1。

最终数组从小到大有序（最大元素依次沉入尾部）。

```
void Heapsort( ElementType A[ ], int N )
{
    int i;
    for ( i = N / 2; i >= 0; i-- )       /* BuildHeap */
        PercDown( A, i, N );
    for ( i = N - 1; i > 0; i-- ) {
        Swap( &A[ 0 ], &A[ i ] );        /* DeleteMax */
        PercDown( A, 0, i );
    }
}
```

注意：下标从 0 开始

课件堆排序中，数组下标从 **0** 开始（而非优先队列章节中的 1）。
对于下标 i 处的节点：

- 左孩子：$2i+1$；右孩子：$2i+2$；父节点：$\lfloor (i-1)/2 \rfloor$。

PercDown 的实现需相应调整。

6.3 复杂度分析

- 建堆：$O(N)$（线性时间建堆定理）。
- 排序：$N-1$ 次 Swap + PercDown，每次 $O(\log N)$，共 $O(N \log N)$。
- 总时间：$T(N) = O(N \log N)$。
- 空间：$O(1)$（原地排序）。

定理（平均比较次数）

堆排序对 N 个不同元素的随机排列的**平均比较次数**为：

$$2N \log N - O(N \log \log N)$$

重要注记

尽管堆排序的平均时间是所有排序算法中最优的之一，但在实践中，它通常**慢于使用 Sedgewick 增量序列的希尔排序**。

原因：堆排序的内存访问模式**缺乏局部性**（cache 命中率低），而希尔排序对缓存更友好。

6.4 堆排序特点总结

性质	结论	原因
时间复杂度	$O(N \log N)$	建堆 $O(N)$ + 排序 $O(N \log N)$
空间复杂度	$O(1)$	原地排序，无需辅助数组
稳定性	**不稳定**	Swap 可能改变相等元素的相对顺序

7 归并排序 (Mergesort)

7.1 步骤一：合并两个有序表

归并 (Merge) 操作将两个已排序的子数组合并为一个有序数组，时间复杂度为 $O(N)$：

```
/* Lpos = start of left half, Rpos = start of right half */
void Merge( ElementType A[ ], ElementType TmpArray[ ],
            int Lpos, int Rpos, int RightEnd )
{
    int i, LeftEnd, NumElements, TmpPos;
    LeftEnd = Rpos - 1;
    TmpPos = Lpos;
    NumElements = RightEnd - Lpos + 1;
    while ( Lpos <= LeftEnd && Rpos <= RightEnd )
        if ( A[ Lpos ] <= A[ Rpos ] )
            TmpArray[ TmpPos++ ] = A[ Lpos++ ];
        else
            TmpArray[ TmpPos++ ] = A[ Rpos++ ];
    while ( Lpos <= LeftEnd )             /* 复制左半剩余 */
        TmpArray[ TmpPos++ ] = A[ Lpos++ ];
    while ( Rpos <= RightEnd )            /* 复制右半剩余 */
        TmpArray[ TmpPos++ ] = A[ Rpos++ ];
```

```
    for ( i = 0; i < NumElements; i++, RightEnd-- )  /* 写回 */
        A[ RightEnd ] = TmpArray[ RightEnd ];
}
```

代码关键点

- 双指针 Lpos/Rpos 分别扫描左右半段，每次取较小者放入 TmpArray。
- 一方耗尽后，将另一方剩余元素直接复制到 TmpArray。
- 最后从 TmpArray 写回原数组 A（逆序写回以避免额外变量）。
- 归并操作是**稳定**的：左半 $\leq$ 右半时优先取左半，保持相等元素的相对顺序。

7.2 步骤二：递归归并排序（MSort）

```
void MSort( ElementType A[ ], ElementType TmpArray[ ],
            int Left, int Right )
{
    int  Center;
    if ( Left < Right ) {
        Center = ( Left + Right ) / 2;
        MSort( A, TmpArray, Left, Center );       /* T(N/2) */
        MSort( A, TmpArray, Center + 1, Right );  /* T(N/2) */
        Merge( A, TmpArray, Left, Center + 1, Right ); /* O(N) */
    }
}

void Mergesort( ElementType A[ ], int N )
{
    ElementType *TmpArray;
    TmpArray = malloc( N * sizeof( ElementType ) );
    if ( TmpArray != NULL ) {
        MSort( A, TmpArray, 0, N - 1 );
        free( TmpArray );
    } else
        FatalError( "No space for tmp array!!!" );
}
```

辅助数组的空间策略

- 若在每次 Merge 调用**内部**局部声明临时数组，则空间复杂度为 $S(N) = O(N \log N)$（每层递归都需要线性空间，共 $\log N$ 层）。
- 正确做法：在 `Mergesort` 中**一次性分配**整个 TmpArray，作为参数传递给每层递归，使得 $S(N) = O(N)$。

7.3 步骤三：复杂度分析

归并排序的递推式为：

$$T(1) = O(1), \quad T(N) = 2T\left(\frac{N}{2}\right) + O(N)$$

逐层展开：

$$\begin{aligned} T(N) &= 2T\left(\frac{N}{2}\right) + O(N) \\ &= 2^k T\left(\frac{N}{2^k}\right) + k \cdot O(N) \\ &= N \cdot T(1) + \log N \cdot O(N) \\ &= O(N + N \log N) = O(N \log N) \end{aligned}$$

时间复杂度结论

归并排序在**所有情况**（最优、平均、最坏）下时间复杂度均为 $O(N \log N)$，这是所有基于比较的排序算法的渐进下界。

7.4 迭代版本 (Iterative Mergesort)

核心思路：自底向上合并

递归版归并排序是”自顶向下”拆分再合并；迭代版反过来，从最小的段开始”自底向上”合并：

迭代归并排序算法框架

外层循环控制**当前段长** `size`，初始为 1，每轮翻倍：
内层循环沿数组从左到右，每次取相邻两段（左段 $[\texttt{Lpos}, \texttt{Lpos} + \texttt{size} - 1]$，右段 $[\texttt{Lpos} + \texttt{size}, \texttt{RightEnd}]$）调用一次 Merge。
重复 $\lceil \log N \rceil$ 轮，直到段长覆盖整个数组。

代码实现

```
void Mergesort( ElementType A[ ], int N )
```

```
{
    ElementType *TmpArray;
    int  size, Lpos, Rpos, RightEnd;

    TmpArray = malloc( N * sizeof( ElementType ) );
    if ( TmpArray == NULL )
        FatalError( "No space for tmp array!!!" );

    for ( size = 1; size < N; size *= 2 ) {
        /* 每轮：将相邻的两个长为 size 的段合并 */
        for ( Lpos = 0; Lpos + size < N; Lpos += 2 * size ) {
            Rpos = Lpos + size;
            RightEnd = ( Lpos + 2 * size - 1 < N - 1 ) ?
                       Lpos + 2 * size - 1 : N - 1;
            Merge( A, TmpArray, Lpos, Rpos, RightEnd );
        }
    }
    free( TmpArray );
}
```

代码关键点

- 内层循环条件 `Lpos + size < N`：保证右半段至少有一个元素才调用 Merge；若最后只剩一个孤立段（无右半），直接跳过，它已有序。
- `RightEnd = min(Lpos + 2*size - 1, N - 1)`：当 N 不是 2 的幂时，最后一次 Merge 的右半段可能不足 `size` 个元素，用 `N-1` 截断，避免越界。
- 复用递归版的 `Merge` 函数，无需任何修改。
- 无递归调用栈，空间复杂度为 $O(N)$（仅 TmpArray）。

示例追踪（$N = 8$）

输入：[8, 3, 2, 9, 7, 1, 5, 4]

轮次	段长	合并操作	数组状态
初始	—	—	[8, 3, 2, 9, 7, 1, 5, 4]
第 1 轮	1	(8,3), (2,9), (7,1), (5,4)	[3, 8, 2, 9, 1, 7, 4, 5]
第 2 轮	2	(3–8, 2–9), (1–7, 4–5)	[2, 3, 8, 9, 1, 4, 5, 7]
第 3 轮	4	(2–3–8–9, 1–4–5–7)	[1, 2, 3, 4, 5, 7, 8, 9]

N 不是 2 的幂时的边界处理（$N = 6$）

输入：[5, 3, 8, 1, 9, 2]

轮次	段长	合并操作	数组状态
初始	—	—	[5, 3, 8, 1, 9, 2]
第 1 轮	1	(5,3), (8,1), (9,2)	[3, 5, 1, 8, 2, 9]
第 2 轮	2	(3−5, 1−8)；[2,9] 孤立段跳过	[1, 3, 5, 8, 2, 9]
第 3 轮	4	(1−3−5−8, 2−9)，右半只 2 元素	[1, 2, 3, 5, 8, 9]

第 2 轮中，`Lpos=4`，`Lpos+size=6`，不满足 `< N=6`，内层循环退出，段 [2,9] 保持原状。第 3 轮中，`RightEnd = min(7, 5) = 5`，Merge 自动截断到数组末尾。

与递归版的对比

特性	递归版	迭代版
实现方式	自顶向下，先拆后合	自底向上，直接合并
递归栈	$O(\log N)$ 栈帧	无
辅助数组	$O(N)$（一次性分配）	$O(N)$（一次性分配）
总空间	$O(N)$	$O(N)$
代码复杂度	更直观	边界处理稍复杂

7.5 归并排序特点总结

性质	结论	原因
时间复杂度	$O(N \log N)$（所有情况）	递推 $T(N) = 2T(N/2) + O(N)$
空间复杂度	$O(N)$	需要辅助数组 TmpArray
稳定性	**稳定**	Merge 中左半 $\leq$ 右半时优先取左半

重要注记

归并排序需要**线性额外内存**，且复制数组操作较慢，因此：

- **内部排序**：几乎不使用归并排序（实践中通常比快速排序慢）。
- **外部排序 (External Sorting)**：归并排序是首选——当数据量超过内存大小、需要借助磁盘排序时，归并排序的顺序读写模式对磁盘极为友好，是外部排序算法的核心思路。

8 快速排序 (Quicksort)

8.1 算法概述

快速排序定位

快速排序是**实践中已知最快的排序算法**，平均时间复杂度 $O(N \log N)$，且常数因子极小、缓存友好。其核心思路为：

1. 从数组中选取一个**基准元素 (pivot)**；
2. 将数组分成两部分：小于等于 pivot 的元素（左）和大于等于 pivot 的元素（右）；
3. 对两部分**递归**排序。

基准元素在分区后即处于正确位置，**不再移动**。

```
void Quicksort ( ElementType A[ ], int N )
{
    if ( N < 2 ) return;
    pivot = pick any element;
    partition S into:
        S1 = { a in S - {pivot} | a <= pivot }
        S2 = { a in S - {pivot} | a >= pivot }
    return Quicksort(S1) + {pivot} + Quicksort(S2);
}
```

课件结论：The best case $T(N) = O(N \log N)$，**the pivot is placed at the right place once and for all**。

8.2 基准元素选取 (Picking the Pivot)

错误方法：取首元素

`Pivot = A[Left]` 是常见错误：

- **最坏情况**：当输入已经有序（升序或降序）时，每次分区只去掉一个元素，退化为 $O(N^2)$，毫无意义。

随机选取

随机选取 `Pivot = A[random(Left, Right)]` 理论上安全，但**随机数生成代价较高**，实践中不常用。

三数取中法 (Median-of-Three Partitioning)

推荐方法：三数取中

取左端 `A[Left]`、中间 `A[Center]`、右端 `A[Right]` 三个位置的**中位数**作为 pivot：

$$\texttt{Pivot} = \text{median}(A[\texttt{Left}],\ A[\texttt{Center}],\ A[\texttt{Right}])$$

- 消除了有序输入的坏情况；
- 相比随机选取，实际运行时间约减少 5%；
- 实现时同时将三元素排好序（$A[\texttt{Left}] \le A[\texttt{Center}] \le A[\texttt{Right}]$），再把 pivot 藏到 $A[\texttt{Right}-1]$，简化分区边界处理。

8.3 分区策略 (Partitioning Strategy)

分区使用双指针 i（从左向右）和 j（从右向左），找到大于 pivot 和小于 pivot 的元素后互换。

关键问题：遇到与 pivot 相等的元素（`key == pivot`）时，i 和 j 应如何处理?

策略	对相等元素的操作	后果
都不停止	跳过相等元素，不交换	全相等输入退化为 $O(N^2)$
都停止并交换	遇到相等即停，互换位置	产生冗余交换，但分区均衡，$O(N \log N)$

结论

应让 i 和 j 都在相等元素处停下并交换。虽然产生了若干冗余交换，但保证了分区均衡性——即使输入全为相同元素，每次仍能平均分为两半，保持 $O(N \log N)$ 的性能。

8.4 小数组优化

对规模较小的子数组（$N \le 20$），快速排序因递归开销大而**慢于插入排序**。

解决方案：设置截断阈值 `Cutoff`（通常取 10），当子数组规模 $\le$ `Cutoff` 时，改用插入排序。

8.5 完整代码实现

```
/* Return median of Left, Center, Right;
   arrange so A[Left]<=A[Center]<=A[Right],
   hide pivot at A[Right-1]                */
ElementType MedianOfThree( ElementType A[], int Left, int Right )
{
```

```
    int Center = ( Left + Right ) / 2;
    if ( A[ Left ] > A[ Center ] )
        Swap( &A[ Left ], &A[ Center ] );
    if ( A[ Left ] > A[ Right ] )
        Swap( &A[ Left ], &A[ Right ] );
    if ( A[ Center ] > A[ Right ] )
        Swap( &A[ Center ], &A[ Right ] );
    /* A[Left] <= A[Center] <= A[Right] */
    Swap( &A[ Center ], &A[ Right - 1 ] ); /* Hide pivot */
    return A[ Right - 1 ];                 /* Return pivot */
}

void Qsort( ElementType A[], int Left, int Right )
{
    int  i, j;
    ElementType  Pivot;

    if ( Left + Cutoff <= Right ) {
        Pivot = MedianOfThree( A, Left, Right );
        i = Left; j = Right - 1;
        for ( ; ; ) {
            while ( A[ ++i ] < Pivot ) { }
            while ( A[ --j ] > Pivot ) { }
            if ( i < j )
                Swap( &A[ i ], &A[ j ] );
            else
                break;
        }
        Swap( &A[ i ], &A[ Right - 1 ] ); /* Restore pivot */
        Qsort( A, Left, i - 1 );
        Qsort( A, i + 1, Right );
    } else
        InsertionSort( A + Left, Right - Left + 1 );
}

void Quicksort( ElementType A[], int N )
{
    Qsort( A, 0, N - 1 );
}
```

代码关键点

- **MedianOfThree**：三次比较排好 Left/Center/Right，将中位数（pivot）藏到 $A[\texttt{Right}-1]$；实际分区范围缩减为 $[\texttt{Left}+1, \texttt{Right}-2]$。
- **分区循环**：`++i` 先移动再比较（跳过已确认 $\leq$ pivot 的 `A[Left]`），`--j` 同理；两指针在相等元素处都停下，执行交换。
- **恢复 pivot**：分区结束时 i 指向第一个 $\geq$ pivot 的位置，将 $A[i]$ 与藏在 $A[\texttt{Right}-1]$ 的 pivot 互换，即完成定位。
- **小数组截断**：`Left + Cutoff > Right` 时调用插入排序，避免递归开销。

8.6 复杂度分析

设分区后左子数组有 i 个元素，递推式为：

$$T(N) = T(i) + T(N-i-1) + cN$$

情况	条件	时间复杂度
最坏情况	pivot 始终为最小/最大值（$i=0$）	$T(N) = T(N-1) + cN \Rightarrow O(N^2)$
最优情况	pivot 始终为中值（$i = \lfloor N/2 \rfloor$）	$T(N) = 2T(N/2) + cN \Rightarrow O(N \log N)$
平均情况	对所有 i 均匀取平均	$T(N) = O(N \log N)$

实践说明

采用三数取中法后，有序或逆序输入的最坏情况几乎不会出现。快速排序的实践性能优于堆排序，因为其访存模式对 CPU 缓存更友好（局部性更好）。

8.7 选择问题 (The Selection Problem)

问题定义

给定 N 个元素和整数 k，找出**第 k 大**的元素。

快速排序的分区思路可直接用于此问题（QuickSelect）：每次分区后，只需递归处理**包含第 k 大元素的那一半**，另一半无需排序。

- 平均时间复杂度：$O(N)$；
- 最坏情况：$O(N^2)$（可通过随机化规避）。

8.8 快速排序特点总结

性质	结论	原因
时间复杂度（平均）	$O(N \log N)$	平均分区，递推同归并排序
时间复杂度（最坏）	$O(N^2)$	pivot 始终最小/最大，可通过三数取中规避
空间复杂度	$O(\log N)$	递归栈平均深度，最坏 $O(N)$
稳定性	**不稳定**	分区交换可改变相等元素的相对顺序

9 大结构排序 (Table Sort)

9.1 问题背景

当待排序的**记录（结构体）非常大**时，每次交换需要复制大量数据，代价极高。

解决方案（间接排序）：不直接交换记录，而是额外维护一个**下标表（table）**，只对表中的下标进行排序。

- 向记录中加入指针字段，排序时只交换指针（或下标），不移动真实数据。
- 排序后有序序列为：`list[table[0]]`, `list[table[1]]`, ..., `list[table[`$N-1$`]]`。

9.2 物理重排（利用置换环）

若需将记录**实际移动**到排好序的位置，可利用置换的**环结构 (disjoint cycles)** 高效完成：

核心性质

任何置换都可以分解为若干不相交的环 (disjoint cycles)。
对长度为 k 的环，只需：

1. 将环起始元素暂存到 `temp`（1 次移动）；
2. 沿环依次将每个元素移入前一位置（$k-1$ 次移动）；
3. 将 `temp` 放入最后空出的位置（1 次移动）。

共 $k+1$ 次记录移动，完成一个长度为 k 的环的物理重排。

9.3 复杂度

- **最坏情况**：$\lfloor N/2 \rfloor$ 个长度为 2 的环，每个需 3 次移动，共 $3N/2$ 次记录移动。
- **总时间**：$T = O(m \cdot N)$，其中 m 为单条记录的字节大小。

10 排序算法的通用下界 (General Lower Bound for Sorting)

定理（基于比较的排序通用下界）

任何**仅通过比较**来排序的算法，其最坏情况时间复杂度不低于 $\Omega(N \log N)$：

$$T_{\text{worst}}(N) = \Omega(N \log N)$$

10.1 决策树证明

将排序算法的每次” 比较-分支” 操作建模为一棵**决策树 (decision tree)**：

- 每个**内部节点**表示一次比较（如 $K_0 \le K_1$?），两个分支对应” 是”/” 否”。
- 每个**叶节点**表示一种排列结果（完成排序的输出）。

关键推导：

1. 对 N 个不同元素排序，有 $N!$ 种不同的排列结果，故决策树**至少有 $N!$ 个叶节点**。
2. 高度为 k 的二叉树最多有 2^{k-1} 个叶节点：

$$N! \le 2^{k-1} \implies k \ge \log_2(N!) + 1$$

3. 由于 $N! \ge \left(\frac{N}{2}\right)^{N/2}$，取对数：

$$\log_2 N! \ge \frac{N}{2} \log_2 \frac{N}{2} = \Omega(N \log_2 N)$$

4. 因此最坏情况下比较次数：$T(N) = k \ge c \cdot N \log_2 N$。

10.2 与第三节逆序对下界的关系

定理	范围	结论
逆序对下界（第 3 节）	仅限相邻元素交换算法	平均情况 $\Omega(N^2)$
决策树下界（本节）	所有基于比较的排序	最坏情况 $\Omega(N \log N)$

两个下界互相独立补充：前者说明” 仅交换相邻元素” 的算法**平均性能差**；后者说明任何基于比较的算法**最坏性能不可能优于** $O(N \log N)$，归并、堆、快速排序均已达到此界。

11 桶排序与基数排序 (Bucket Sort & Radix Sort)

11.1 桶排序 (Bucket Sort)

场景与思路

当元素取值范围 M 已知且有限时，可以**不通过比较**，直接按值分桶：

- 为每个可能的值建立一个**桶（链表）**；
- 将每个元素放入对应的桶；
- 顺序遍历所有桶，输出各桶内元素。

```
Algorithm {
    initialize count[ ];
    while ( read in a student's record )
        insert to list count[ grade ];
    for ( i = 0; i < M; i++ )
        output list count[ i ];
}
```

时间复杂度：$T(N, M) = O(M + N)$。

当 $M = O(N)$ 时实现线性时间排序，**突破基于比较的排序** $\Omega(N \log N)$ **下界**（因为它不依赖比较）。

问题：若 $M \gg N$（如 $N = 10$，$M = 1000$），初始化和遍历 M 个桶的代价远超排序本身，效率极低。

11.2 基数排序 (Radix Sort)

基数排序通过**多趟桶排序**解决 $M \gg N$ 的问题：将每个元素按**一位数字**分桶，多趟完成整体排序。

$$T = O(P \cdot (N + B))$$

其中 P 为趟数（位数），B 为每趟的桶数（基数），N 为元素个数。

示例：LSD（最低位优先）基数排序

输入：$0, 1, 8, 27, 64, 125, 216, 343, 512, 729$（10 个完全立方数，范围 0–999）

按**从低位到高位**（个位 $\rightarrow$ 十位 $\rightarrow$ 百位）依次做桶排序（$B = 10$ 个桶）：

趟次	按哪一位排序	排序结果
第 1 趟	个位	0, 1, 512, 343, 64, 125, 216, 27, 8, 729
第 2 趟	十位	0, 1, 8, 512, 216, 125, 27, 729, 343, 64
第 3 趟	百位	0, 1, 8, 27, 64, 125, 216, 343, 512, 729

3 趟 $\times$ 10 桶：$T = O(3 \times (10 + 10))$，远优于直接用 1000 个桶的 $O(1000 + 10)$。

多键词典排序（MSD vs LSD）

记录有 r 个键 $K^0, K^1, \ldots, K^{r-1}$（$K^0$ 最高位，K^{r-1} 最低位）。

策略	MSD 优先（最高位优先）	LSD 优先（最低位优先）
排序顺序	先排 K^0，再递归排各桶	先排 K^{r-1}，依次到 K^0
辅助需求	每个桶独立排序	每趟**必须**使用稳定排序
典型场景	字符串字典序	数字多键排序（如扑克牌按花色 + 面值排序）

LSD 为何需要稳定排序?

LSD 策略中，高位趟次必须**保留低位已排好的相对顺序**。若高位趟次使用不稳定排序，同一桶内的元素顺序会被打乱，导致低位的有序性丢失。因此每趟必须使用稳定的桶排序，确保同一桶内元素按进入顺序输出。

结论：LSD 并不总是比 MSD 快；MSD 对字符串等有提前终止优势，LSD 对固定位数的数字处理更均匀。

11.3 桶排序与基数排序特点总结

算法	时间复杂度	适用场景
桶排序	$O(M + N)$	取值范围 M 小（$M = O(N)$）
基数排序（LSD）	$O(P(N + B))$	多位数字，$M \gg N$

关键优势

桶排序和基数排序均**不基于比较**，可以突破 $\Omega(N \log N)$ 下界，在特定场景下实现**线性时间**排序。

12 全章排序算法横向对比

算法	最优	平均	最坏	空间	稳定	特点
插入排序	$O(N)$	$O(N^2)$	$O(N^2)$	$O(1)$	是	近序输入最优
选择排序	$O(N^2)$	$O(N^2)$	$O(N^2)$	$O(1)$	否	交换次数最少
希尔排序	$O(N \log N)$	$O(N^{7/6})^*$	$O(N^{4/3})^*$	$O(1)$	否	简单高效
堆排序	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(1)$	否	缓存不友好
归并排序	$O(N \log N)$	$O(N \log N)$	$O(N \log N)$	$O(N)$	**是**	外排首选
快速排序	$O(N \log N)$	$O(N \log N)$	$O(N^2)^{**}$	$O(\log N)$	否	实践最快
桶/基数排序	$O(N+M)$	$O(P(N+B))$	—	$O(N+B)$	是	非比较，线性

$*$ 使用 Sedgewick 增量序列。 $**$ 三数取中法后实践中几乎不出现。

选型建议

- **通用内部排序**：通常选用快速排序（三数取中 + 小数组截断）或 Sedgewick 希尔排序。
- **需要稳定性**：归并排序（或 Timsort 等混合变体）。
- **外部排序**：归并排序，利用顺序 I/O 的优势。
- **近乎有序的输入**：插入排序（$O(N+I)$，I 为逆序对数）。
- **需保证最坏情况**：堆排序（理论最优但实践受限于缓存）。
- **取值范围有限、非比较**：桶排序（$M = O(N)$）或基数排序（多位数字）。
- **大型记录排序**：结合表格排序（间接排序），减少数据移动代价。

13 全章总结

1. **插入排序**把每个新元素插入已排序前缀，精确复杂度为 $O(N+I)$（I 为逆序对数）；对接近有序的输入最友好，是原地稳定排序。
2. **选择排序**每轮从未排序部分找最小元素并交换到前端；比较次数始终 $O(N^2)$（与输入无关），但交换次数至多 $N-1$ 次，适合写操作代价高的场景；不稳定。
3. **简单排序下界**：任何仅通过交换相邻元素排序的算法，平均时间不低于 $\Omega(N^2)$；要突破此下界，必须在一次操作中消除多个逆序对。

4. **希尔排序**通过逐步缩小的增量序列执行插入排序，以" 跳步交换" 突破 $\Omega(N^2)$。增量序列的质量决定性能：Shell 序列最差（$\Theta(N^2)$），Hibbard 序列（$\Theta(N^{3/2})$），Sedgewick 序列最优（$O(N^{4/3})$）。

5. **堆排序**利用最大堆性质，先建堆再依次 DeleteMax，实现原地 $O(N \log N)$ 排序；理论上时间最优，但因缓存不友好而在实践中通常慢于希尔排序。

6. **归并排序**采用" 分而治之", 在所有情况下均为 $O(N \log N)$, 且天然稳定; 代价是需要 $O(N)$ 额外空间，是外部排序的核心算法。

7. **快速排序**实践中最快：三数取中法选 pivot 避免有序输入退化，分区时对相等元素应双指针都停并交换（保证均衡分区），小数组截断改用插入排序；平均 $O(N \log N)$，最坏 $O(N^2)$ 可规避。

8. **大结构排序（表格排序）**：对大型记录只排序下标表（间接排序），再利用置换环进行物理重排；至多 $3N/2$ 次记录移动，总代价 $O(mN)$（m 为记录大小）。

9. **排序通用下界**：任何基于比较的排序，最坏情况至少需要 $\Omega(N \log N)$ 次比较——决策树叶节点数 $\geq N!$，树高 $\geq \log_2 N! = \Omega(N \log N)$。归并、堆、快速排序均达到此界。

10. **桶排序与基数排序**不依赖比较，可突破 $\Omega(N \log N)$ 下界：桶排序 $T = O(M + N)$（取值范围小时）；LSD 基数排序 $T = O(P(N + B))$（多位数字、$M \gg N$ 时）；LSD 每趟必须稳定，以保留低位已有的有序性。

11. 选择排序算法需权衡：时间复杂度、空间约束、稳定性需求，以及实际运行环境（内存局部性、外部存储、数据分布特征）。

"认清逆序对是理解所有比较排序复杂度的钥匙；选择合适的算法需要在时间、空间与稳定性之间权衡。"